

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №6
ГОСТ Р 34.10-2018: электронная цифровая подпись
по дисциплине
«Криптографические методы защиты информации»

Выполнил
студент гр. 5131001/20502

Улановский Г. А

Преподаватель
ассистент ВШК ИКНК

Иванов М. С.

1 ЦЕЛЬ РАБОТЫ

Изучить формирование и проверку электронной цифровой подписи ГОСТ Р 34.10-2018 на эллиптических кривых, реализовать подпись файла, проверку подписи и ASN.1 DER-контейнер подписи для варианта 8.

2 ЗАДАЧИ РАБОТЫ

В ходе выполнения лабораторной работы необходимо:

- реализовать параметры эллиптической кривой варианта 8;
- сформировать закрытый и открытый ключи d и $Q = dP$;
- вычислить хэш сообщения алгоритмом Стрибог-256;
- сформировать подпись ГОСТ Р 34.10-2018 с параметрами r и s ;
- сохранить подпись в ASN.1 DER-контейнере по структуре приложения

Г;

- проверить корректную подпись, изменённое сообщение и испорченную подпись;
- подготовить данные, листинги и команды для отчёта и защиты.

3 ХОД РАБОТЫ

В ходе работы была разработана учебная программа на языке Python. Она содержит модули ручной арифметики эллиптических кривых, хэширования Стрибог, формирования и проверки подписи, кодирования ASN.1 DER-контейнера, CLI-интерфейс и генератор отчётных материалов. Общая структура программы представлена на рисунке 1.

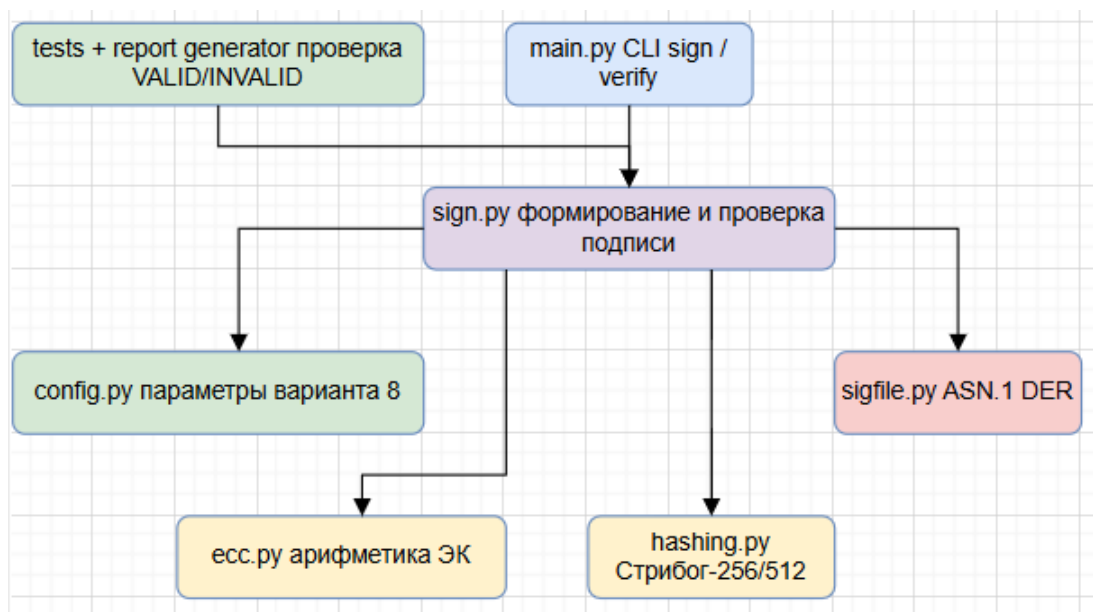


Рисунок 1 – Структура разработанной программы Lab6

3.1 Теоретические сведения

3.1.1 Эллиптические кривые над конечным полем

Эллиптическая кривая над конечным полем F_p задаётся уравнением $y^2 = x^3 + a * x + b \pmod{p}$. Все вычисления координат выполняются по модулю простого числа p , поэтому множество точек кривой конечно. К группе точек добавляется точка на бесконечности, являющаяся нейтральным элементом.

Сложение двух разных точек выполняется через наклон прямой, проходящей через эти точки. Удвоение точки использует касательную к кривой. Скалярное умножение kP реализуется двоичным методом double-and-add.

3.1.2 Ключи и подпись ГОСТ Р 34.10-2018

Закрытый ключ d выбирается как число $0 < d < q$. Открытый ключ вычисляется как $Q = dP$, где P — базовая точка подгруппы порядка q . Перед подписью сообщение хэшируется алгоритмом Стрибог, а значение хэша интерпретируется как целое число α .

Формирование подписи выполняется по шагам: $\alpha = H(M)$, $e = \alpha \bmod q$, выбирается случайное k , вычисляется $C = kP$, $r = x_C \bmod q$, $s = (r * d + k * e) \bmod q$. Если r или s равны нулю, выбирается новое k . Схема формирования подписи показана на рисунке 2.

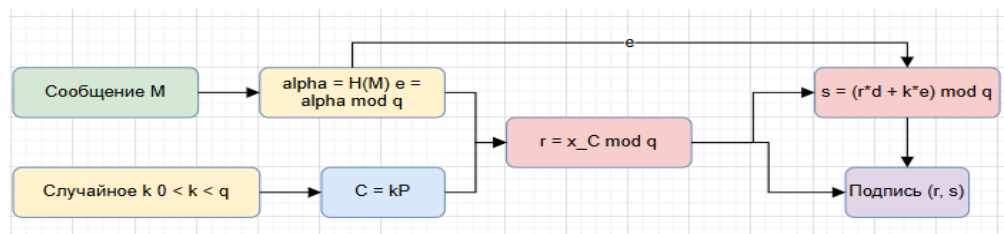


Рисунок 2 – Формирование подписи ГОСТ Р 34.10-2018

Проверка подписи выполняется через значения $v = e^{-1} \bmod q$, $z_1 = s * v \bmod q$ и $z_2 = -r * v \bmod q$. Затем вычисляется $C = z_1 * P + z_2 * Q$ и проверяется равенство $R = x_C \bmod q$ и $R = r$. Схема проверки приведена на рисунке 3.

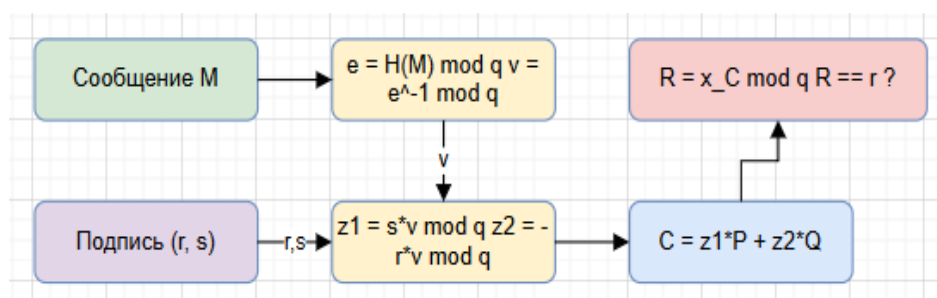


Рисунок 3 – Проверка подписи ГОСТ Р 34.10-2018

3.2 Описание реализации

Реализация разделена на несколько модулей. `config.py` хранит параметры варианта 8, `ecc.py` реализует арифметику точек эллиптической кривой, `hashing.py`

вычисляет хэши Стрибог через библиотеку gostcrypto, sigfile.py кодирует и декодирует ASN.1 DER-контейнер, sign.py реализует формирование и проверку подписи, main.py предоставляет CLI-команды sign и verify.

3.3 Параметры варианта 8 и ключи

В работе использовалась кривая варианта 8. Основные параметры приведены в таблице 1.

Таблица 1 – Параметры эллиптической кривой варианта 8

Параметр	Значение
curve_id	gost3410-2018-var8
p	5789604462895871863121302827551841132847614959978977073875 7218840632915517411
a	-1
b	4451642394801966182542081392796559234167583927084986044186 1020678502941837466
q	2894802231447935931560651413775920566423683202323162803587 1193493020981068937
P.x	2323576058601956664720966708045726308916627824741707729836 708887517232685058
P.y	2077230201105399139012743526229771501036701838313146783160 9444907978987653753

Для демонстрационного запуска был использован закрытый ключ $d = 123456789$. Открытый ключ вычислен как $Q = dP$. Значения открытого ключа приведены в таблице 2.

Таблица 2 – Закрытый и открытый ключи

Параметр	Значение
d	123456789
Q.x	2543045702302399326200225425985500535232011684917708293250 6990180192002774338
Q.y	5350453908189687268723302972595977287424103151734505379892 3234020870867244235

3.4 Хэширование сообщения

Исходный файл содержит демонстрационное сообщение для подписи. Текст сообщения приведён в листинге 1.

Листинг 1 – Текст подписываемого сообщения

Лабораторная работа 6. ГОСТ Р 34.10-2018. Демонстрационное сообщение для подписи, вариант 8.

Для сообщения был вычислен хэш Streebog-256. Полученные значения α и e приведены в таблице 3.

Таблица 3 – Хэш-образ сообщения и значение e

Параметр	Значение
Первые байты файла	d0 9b d0 b0 d0 b1 d0 be d1 80 d0 b0 d1 82 d0 be d1 80 d0 bd d0 b0 d1 8f 20 d1 80 d0 b0 d0 b1 d0 be d1 82 d0 b0 20 36 2e 20 d0 93 d0 9e d0 a1 d0 a2 20 d0 a0 20 33 34 2e 31 30 2d 32 30 31 38 2e
Streebog-256 digest	80cc787f5cf5b62a887c43f89c091cb9056e1ceb65e7da1b04442638f511037a
$\alpha = \text{int}(\text{hash})$	58257313063137540428158356925056269847154052745756832099068785421798033392506
$e = \alpha \bmod q$	361268434178821796945328649537858518680388699293576027326398435756071254632

3.5 Формирование подписи

После вычисления e была сформирована подпись ГОСТ Р 34.10-2018. В отчётном генераторе для воспроизводимости используется демонстрационное значение k . Полученные значения r и s приведены в таблице 4.

Таблица 4 – Значения подписи r и s

Параметр	Значение
r	10485874236376302654048979674314657321497825192062511225776966478011850356911
s	4476988221943700431939203327141955944938407055758235851522990464512984490822

3.6 ASN.1 DER-контейнер подписи

Подпись сохраняется в ASN.1 DER-контейнере по структуре приложения Г. Контейнер содержит идентификатор алгоритма, метку ключа, открытый ключ Q , параметры кривой, значения подписи r и s , а также пустую последовательность параметров файла. Структура контейнера показана на рисунке 4.

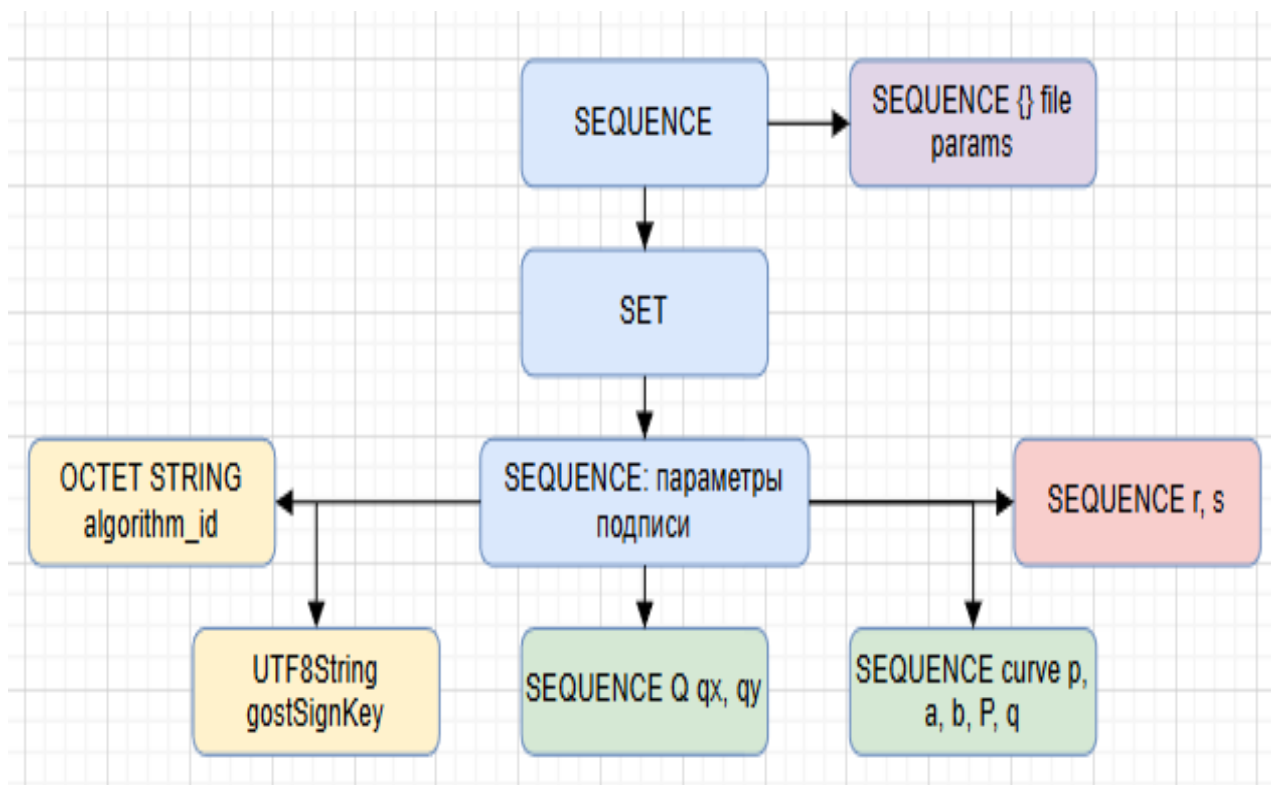


Рисунок 4 – Структура ASN.1 DER-контейнера подписи

Первые 96 байт файла подписи представлены в листинге 2.

Листинг 2 – Первые 96 байт ASN.1 DER-контейнера подписи

0000:	30	82	01	60	31	82	01	5a	30	82	01	56	04	04	80	06
0010:	07	00	0c	0b	67	6f	73	74	53	69	67	6e	4b	65	79	30
0020:	44	02	20	38	39	20	ec	0b	c1	e3	cc	51	e4	59	a5	83
0030:	b5	46	57	a1	63	33	e7	85	06	eb	f4	b0	26	c3	bb	04
0040:	1c	15	42	02	20	76	4a	7e	f6	85	cd	f6	4c	b3	58	c9
0050:	61	3d	6e	b6	c3	e7	4d	52	07	eb	cc	ef	fd	61	09	8f

Основные поля, извлечённые из контейнера, приведены в таблице 5.

Таблица 5 – Основные поля ASN.1 DER-контейнера подписи

Параметр	Значение
algorithm_id_hex	80 06 07 00
key_label	gostSignKey
hash_alg	streebog256
curve_id	gost3410-2018-var8
Q.x	254304570230239932620022542598550053523201168491770 82932506990180192002774338
Q.y	535045390818968726872330297259597728742410315173450 53798923234020870867244235
p	578960446289587186312130282755184113284761495997897 70738757218840632915517411
a	-1
b	445164239480196618254208139279655923416758392708498 60441861020678502941837466
P.x	232357605860195666472096670804572630891662782474170 7729836708887517232685058
P.y	207723020110539913901274352622977150103670183831314 67831609444907978987653753
q	289480223144793593156065141377592056642368320232316 28035871193493020981068937
r	104858742363763026540489796743146573214978251920625 11225776966478011850356911
s	447698822194370043193920332714195594493840705575823 5851522990464512984490822
file_params	empty SEQUENCE

3.7 Проверка подписи

Корректность реализации проверялась на трёх случаях: исходное сообщение должно проходить проверку, изменённое сообщение должно отклоняться, испорченная подпись также должна отклоняться. Итоговая схема проверки показана на рисунке 5.

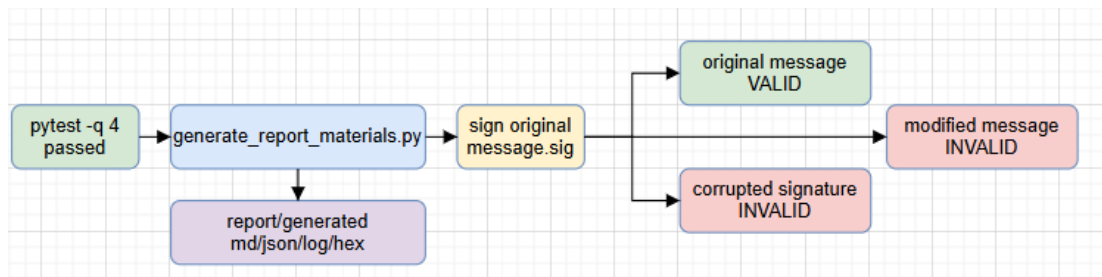


Рисунок 5 – Итоговая схема проверки подписи

Журнал проверки приведён в листинге 3.

Листинг 3 – Журнал проверки подписи

```

Verification log
original message: VALID
modified message: INVALID
corrupted signature: INVALID
  
```

Результаты автоматического запуска тестов и генерации отчётных материалов приведены на рисунке 6.

```

(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI> cd C:\Users\Georgul\Documents\8_sem\KMZI\Lab6
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6>
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6> pytest -q
..... [100%]
4 passed in 0.41s
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6> python scripts\generate_report_materials.py
Generated Lab6 report materials:
report/generated/report_data.md
report/generated/report_data.json
report/generated/verification_log.txt
report/generated/hex_dump.md
report/generated/theory.md
report/generated/control_questions.md
report/generated/stand_commands.md
report/generated/full_report_draft.md
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6> Get-Content .\report\generated\verification_log.txt -Encoding UTF8
Verification log
original message: VALID
modified message: INVALID
corrupted signature: INVALID
(.venv) PS C:\Users\Georgul\Documents\8_sem\KMZI\Lab6>
  
```

Рисунок 6 – Проверка тестов и журнала выполнения ЛР6

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Преимущества криптосистем на эллиптических кривых по сравнению с другими криптосистемами?

Основное преимущество криптосистем на эллиптических кривых состоит в том, что они обеспечивают сопоставимый уровень криптографической стойкости при меньшей длине ключа. Например, для достижения уровня стойкости, аналогичного RSA с большим модулем, в эллиптических схемах можно использовать существенно меньшие параметры. Это уменьшает объём ключевой информации, подписи и передаваемых данных.

В практических системах это особенно важно для устройств с ограниченными ресурсами: смарт-карт, мобильных устройств, встроенных систем и сетевых протоколов. Меньший размер ключей снижает требования к памяти, ускоряет передачу данных и уменьшает нагрузку на канал связи. Кроме того, операции над точками эллиптической кривой при правильно выбранных параметрах могут быть эффективнее операций с очень большими RSA-модулями.

С точки зрения безопасности стойкость эллиптических криптосистем основана на сложности задачи дискретного логарифмирования в группе точек эллиптической кривой. При корректном выборе кривой, базовой точки и порядка подгруппы эта задача считается вычислительно сложной. Поэтому ГОСТ Р 34.10-2018 использует именно эллиптические кривые для построения электронной подписи.

4.2 Почему в ГОСТ Р 34.10-2018 требуется $\#E(F_p) \neq p$?

Условие $\#E(F_p) \neq p$ связано с исключением так называемых аномальных эллиптических кривых. Здесь $\#E(F_p)$ обозначает число точек эллиптической кривой над конечным полем F_p . Если число точек кривой совпадает с характеристикой поля p , такая кривая имеет специальные алгебраические свойства, которые делают её небезопасной для криптографического применения.

Для аномальных кривых известны атаки, которые позволяют свести задачу дискретного логарифмирования на эллиптической кривой к более простой задаче в конечном поле. В результате задача нахождения закрытого ключа d по открытому ключу $Q = dP$ становится существенно проще, чем должна быть в корректно выбранной эллиптической группе.

Именно поэтому стандарт требует исключать случай $\#E(F_p) = p$. Это требование является частью проверки корректности параметров кривой. Оно гарантирует, что выбранная кривая не относится к классу заведомо слабых кривых и не допускает известных специальных атак, нарушающих стойкость схемы электронной подписи.

4.3 Как злоумышленник мог бы подделать сообщение и подпись, если умеет обращаться хэш-функцию?

При проверке электронной подписи проверяется не само сообщение напрямую, а его хэш-образ. В ГОСТ Р 34.10-2018 сообщение M сначала обрабатывается хэш-функцией, после чего полученное значение $\alpha = H(M)$ используется в алгоритме проверки подписи. Следовательно, безопасность подписи зависит не только от сложности задачи дискретного логарифмирования, но и от криптографической стойкости хэш-функции.

Если злоумышленник умеет обращаться хэш-функцию, то есть может по заданному значению хэша подобрать сообщение с таким же хэш-образом, он получает возможность подделки. Например, он может взять уже существующую корректную подпись для некоторого сообщения и попытаться подобрать другое сообщение, имеющее тот же хэш. Так как проверка подписи использует только $H(M)$, новое сообщение будет принято как подписанное, хотя владелец закрытого ключа его не подписывал.

Также опасна возможность подбора сообщения под заранее нужное значение хэша. В этом случае злоумышленник может строить сообщение так, чтобы его хэш участвовал в проверке подписи нужным образом. Поэтому для электронной подписи требуется криптографически стойкая хэш-функция,

обладающая устойчивостью к поиску прообраза, второго прообраза и коллизий. В данной работе используется Стрибог-256, который предназначен для применения в российских криптографических схемах, включая электронную подпись.

4.4 Почему случайное k не должно повторяться в течение срока жизни ключа подписи?

Случайное число k в алгоритме ГОСТ Р 34.10-2018 является критически важным секретным параметром каждой конкретной подписи. Оно используется при вычислении точки $C = kP$, затем значения $r = x_C \bmod q$ и значения $s = (r * d + k * e) \bmod q$. Хотя k не входит в подпись напрямую, оно математически связано с подписью, хэшем сообщения и закрытым ключом d .

Если одно и то же k будет использовано повторно для двух разных сообщений, то между двумя подписями возникнет система сравнений с одним и тем же неизвестным k и одним и тем же закрытым ключом d . Зная две подписи и хэши соответствующих сообщений, злоумышленник может исключить k или сначала восстановить k , а затем вычислить закрытый ключ d . После восстановления d он сможет формировать корректные подписи от имени владельца ключа.

Поэтому k должно быть уникальным, случайным и непредсказуемым для каждой подписи. Повторение k является одной из самых опасных ошибок в реализациях схем электронной подписи на эллиптических кривых. На практике для генерации k используют криптографически стойкий генератор случайных чисел или специальные детерминированные методы генерации k на основе закрытого ключа и хэша сообщения. В учебной реализации для воспроизводимости отчётных данных использовалось фиксированное демонстрационное значение k , но в реальной системе так делать нельзя.

5 ВЫВОДЫ

В ходе лабораторной работы была реализована учебная схема формирования и проверки электронной цифровой подписи ГОСТ Р 34.10-2018 на эллиптических кривых. В программе использованы параметры варианта 8, ручная арифметика точек эллиптической кривой, хэширование Стрибог-256, формирование подписи с вычислением r и s , а также ASN.1 DER-контейнер подписи.

Проверка показала, что подпись корректно подтверждает исходное сообщение и отклоняется при изменении сообщения или порче значения подписи. Это подтверждает корректность связи между хэш-образом сообщения, открытым ключом Q и значениями подписи r и s .

Реализация является учебной и предназначена для демонстрации алгоритма. Для промышленного применения необходимы сертифицированные библиотеки, защищённая генерация случайного параметра k , защита от побочных каналов и безопасное хранение закрытого ключа.

Для обеспечения стойкости к известным методам дискретного логарифмирования параметры эллиптической кривой должны исключать аномальные и слабые кривые, иметь подгруппу большого простого порядка q , корректно выбранную базовую точку P и проверяемый порядок группы. Также необходимо контролировать принадлежность открытого ключа Q выбранной кривой и подгруппе порядка q .

Полный исходный код разработанной программы приведён в приложениях 1–7.

ПРИЛОЖЕНИЕ 1. ЛИСТИНГ ФАЙЛА SRC/ECC.PY

```
"""Manual elliptic curve arithmetic for educational purposes.

Implements operations on curves in short Weierstrass form:
 $y^2 = x^3 + ax + b \pmod{p}$ 
"""

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class ECPoint:
    """Point on an elliptic curve."""

    x: int = 0
    y: int = 0
    is_infinity: bool = False

    @staticmethod
    def infinity() -> "ECPoint":
        """Return the point at infinity."""
        return ECPoint(is_infinity=True)

def mod_inv(value: int, modulus: int) -> int:
    """Compute modular inverse using the extended Euclidean
    algorithm."""
    if modulus <= 0:
        raise ValueError("Modulus must be positive")

    value %= modulus
    if value == 0:
        raise ValueError("Inverse does not exist for zero")

    t, new_t = 0, 1
    r, new_r = modulus, value

    while new_r != 0:
        quotient = r // new_r
        t, new_t = new_t, t - quotient * new_t
        r, new_r = new_r, r - quotient * new_r

    if r != 1:
```

```

        raise ValueError("Inverse does not exist")
    if t < 0:
        t += modulus
    return t

@dataclass(frozen=True)
class EllipticCurve:
    """Elliptic curve with base point and subgroup order."""

    p: int
    a: int
    b: int
    q: int
    base_point: ECPoint
    curve_id: str

    def is_on_curve(self, point: ECPoint) -> bool:
        """Check if point belongs to the curve."""
        if point.is_infinity:
            return True
        left = (point.y * point.y) % self.p
        right = (point.x * point.x * point.x + self.a * point.x +
self.b) % self.p
        return left == right

    def negate(self, point: ECPoint) -> ECPoint:
        """Return additive inverse of a point."""
        if point.is_infinity:
            return point
        return ECPoint(point.x, (-point.y) % self.p)

    def add(self, p1: ECPoint, p2: ECPoint) -> ECPoint:
        """Add two points on the curve."""
        if p1.is_infinity:
            return p2
        if p2.is_infinity:
            return p1

        if p1.x == p2.x and (p1.y + p2.y) % self.p == 0:
            return ECPoint.infinity()

        if p1 == p2:
            return self.double(p1)

        numerator = (p2.y - p1.y) % self.p

```



```

        denominator = (p2.x - p1.x) % self.p
        lam = (numerator * mod_inv(denominator, self.p)) % self.p

        x3 = (lam * lam - p1.x - p2.x) % self.p
        y3 = (lam * (p1.x - x3) - p1.y) % self.p
        return ECPoint(x3, y3)

def double(self, point: ECPoint) -> ECPoint:
    """Double a point on the curve."""
    if point.is_infinity:
        return point
    if point.y % self.p == 0:
        return ECPoint.infinity()

    numerator = (3 * point.x * point.x + self.a) % self.p
    denominator = (2 * point.y) % self.p
    lam = (numerator * mod_inv(denominator, self.p)) % self.p

    x3 = (lam * lam - 2 * point.x) % self.p
    y3 = (lam * (point.x - x3) - point.y) % self.p
    return ECPoint(x3, y3)

def scalar_mul(self, scalar: int, point: ECPoint) -> ECPoint:
    """Multiply point by integer using binary method (double-
and-add)."""
    if scalar == 0 or point.is_infinity:
        return ECPoint.infinity()
    if scalar < 0:
        return self.scalar_mul(-scalar, self.negate(point))

    result = ECPoint.infinity()
    addend = point
    k = scalar

    while k > 0:
        if k & 1:
            result = self.add(result, addend)
            addend = self.double(addend)
            k >>= 1

    return result

```

ПРИЛОЖЕНИЕ 2. ЛИСТИНГ ФАЙЛА SRC/HASHING.PY

```
"""Hashing helpers based on gostcrypto (Streebog)."""

from __future__ import annotations

from pathlib import Path
from typing import Literal

from gostcrypto import gosthash

HashAlg = Literal["streebog256", "streebog512"]


def hash_bytes(data: bytes, hash_alg: HashAlg) -> bytes:
    """Return digest bytes for input data."""
    hasher = gosthash.new(hash_alg)
    hasher.update(bytearray(data))
    return bytes(hasher.digest())


def hash_file(file_path: Path, hash_alg: HashAlg) -> bytes:
    """Read file and hash its contents."""
    data = file_path.read_bytes()
    return hash_bytes(data, hash_alg)


def hash_to_alpha(file_path: Path, hash_alg: HashAlg) -> int:
    """Compute alpha = int(h(M)) for signature steps."""
    digest = hash_file(file_path, hash_alg)
    return int.from_bytes(digest, byteorder="big")
```

ПРИЛОЖЕНИЕ 3. ЛИСТИНГ ФАЙЛА SRC/SIGFILE.PY

```
"""ASN.1 DER signature container for GOST R 34.10-2018 lab work.

Format from Appendix G:

SEQUENCE {
    SET {
        SEQUENCE {
            OCTET STRING      -- algorithm identifier bytes: 80 06
07 00
            UTF8String        -- 'gostSignKey'
            SEQUENCE {        -- public key Q
                INTEGER qx
                INTEGER qy
            }
            SEQUENCE {        -- curve parameters
                SEQUENCE {    -- field params
                    INTEGER p
                }
                SEQUENCE {    -- curve params
                    INTEGER a
                    INTEGER b
                }
                SEQUENCE {    -- base point P
                    INTEGER px
                    INTEGER py
                }
                INTEGER q
            }
            SEQUENCE {        -- signature
                INTEGER r
                INTEGER s
            }
        }
    }
    SEQUENCE {}              -- file params, unused
}
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import asn1
```

```

from src.config import APP_CONFIG

@dataclass(frozen=True)
class SignaturePayload:
    """Signature payload stored inside ASN.1 container."""

    hash_alg: str
    curve_id: str
    qx: int
    qy: int
    p: int
    a: int
    b: int
    px: int
    py: int
    q: int
    r: int
    s: int

def encode_signature(payload: SignaturePayload) -> bytes:
    """Encode signature payload to ASN.1 DER bytes."""
    encoder = asn1.Encoder()
    encoder.start()

    encoder.enter(asn1.Numbers.Sequence)

    encoder.enter(asn1.Numbers.Set)
    encoder.enter(asn1.Numbers.Sequence)

    encoder.write(APP_CONFIG.gost_algorithm_id,
asn1.Numbers.OctetString)
    encoder.write(APP_CONFIG.signature_key_label,
asn1.Numbers.UTF8String)

    # Public key Q
    encoder.enter(asn1.Numbers.Sequence)
    encoder.write(payload.qx, asn1.Numbers.Integer)
    encoder.write(payload.qy, asn1.Numbers.Integer)
    encoder.leave()

    # Curve parameters
    encoder.enter(asn1.Numbers.Sequence)

```

```

# Field parameters
encoder.enter(asn1.Numbers.Sequence)
encoder.write(payload.p, asn1.Numbers.Integer)
encoder.leave()

# Curve coefficients
encoder.enter(asn1.Numbers.Sequence)
encoder.write(payload.a, asn1.Numbers.Integer)
encoder.write(payload.b, asn1.Numbers.Integer)
encoder.leave()

# Base point P
encoder.enter(asn1.Numbers.Sequence)
encoder.write(payload.px, asn1.Numbers.Integer)
encoder.write(payload.py, asn1.Numbers.Integer)
encoder.leave()

# Subgroup order q
encoder.write(payload.q, asn1.Numbers.Integer)
encoder.leave()

# Signature (r, s)
encoder.enter(asn1.Numbers.Sequence)
encoder.write(payload.r, asn1.Numbers.Integer)
encoder.write(payload.s, asn1.Numbers.Integer)
encoder.leave()

encoder.leave()
encoder.leave()

# Unused file params
encoder.enter(asn1.Numbers.Sequence)
encoder.leave()

encoder.leave()

return encoder.output()

def _expect_constructed(decoder: asn1.Decoder, nr: int) -> None:
    """Enter a constructed ASN.1 object of the expected type."""
    tag = decoder.peek()
    if tag is None:
        raise ValueError("Unexpected end of ASN.1 data")
    if tag.nr != nr or tag.typ != asn1.Types.Constructed:

```

```

        raise ValueError(f"Expected constructed ASN.1 tag {nr},
got {tag}")
    decoder.enter()

def _read_primitive(decoder: asn1.Decoder, nr: int):
    """Read a primitive ASN.1 value of the expected type."""
    tag, value = decoder.read()
    if tag is None:
        raise ValueError("Unexpected end of ASN.1 data")
    if tag.nr != nr or tag.typ != asn1.Types.Primitive:
        raise ValueError(f"Expected primitive ASN.1 tag {nr}, got
{tag}")
    return value

def decode_signature(blob: bytes) -> SignaturePayload:
    """Decode ASN.1 DER signature container."""
    decoder = asn1.Decoder()
    decoder.start(blob)

    _expect_constructed(decoder, asn1.Numbers.Sequence)

    _expect_constructed(decoder, asn1.Numbers.Set)
    _expect_constructed(decoder, asn1.Numbers.Sequence)

    algorithm_id = bytes(_read_primitive(decoder,
asn1.Numbers.OctetString))
    if algorithm_id != APP_CONFIG.gost_algorithm_id:
        raise ValueError("Unexpected algorithm identifier")

    key_label = _read_primitive(decoder, asn1.Numbers.UTF8String)
    if not isinstance(key_label, str):
        raise ValueError("Invalid key label type")

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    qx = int(_read_primitive(decoder, asn1.Numbers.Integer))
    qy = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

    _expect_constructed(decoder, asn1.Numbers.Sequence)

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    p = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

```

```

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    a = int(_read_primitive(decoder, asn1.Numbers.Integer))
    b = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    px = int(_read_primitive(decoder, asn1.Numbers.Integer))
    py = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

    q = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    r = int(_read_primitive(decoder, asn1.Numbers.Integer))
    s = int(_read_primitive(decoder, asn1.Numbers.Integer))
    decoder.leave()

    decoder.leave()
    decoder.leave()

    _expect_constructed(decoder, asn1.Numbers.Sequence)
    if not decoder.eof():
        raise ValueError("File params sequence must be empty")
    decoder.leave()

    decoder.leave()

    if not decoder.eof():
        raise ValueError("Unexpected trailing bytes after ASN.1
container")

    return SignaturePayload(
        hash_alg=APP_CONFIG.default_hash_alg,
        curve_id=APP_CONFIG.default_curve_id,
        qx=qx,
        qy=qy,
        p=p,
        a=a,
        b=b,
        px=px,
        py=py,
        q=q,
        r=r,
        s=s,
    )

```

```
def save_signature(path: Path, payload: SignaturePayload) -> None:
    """Write signature payload to file."""
    path.write_bytes(encode_signature(payload))

def load_signature(path: Path) -> SignaturePayload:
    """Read and parse signature payload from file."""
    return decode_signature(path.read_bytes())
```


ПРИЛОЖЕНИЕ 4. ЛИСТИНГ ФАЙЛА SRC/SIGN.PY

```
"""Signing and verification logic for GOST R 34.10-2018
(educational build)."""

from __future__ import annotations

import json
import secrets
from dataclasses import dataclass
from pathlib import Path
from typing import Any, cast

from src.config import APP_CONFIG, CurveParams
from src.ecc import ECPoint, EllipticCurve, mod_inv
from src.hashing import HashAlg, hash_to_alpha
from src.sigfile import SignaturePayload, load_signature,
save_signature

@dataclass(frozen=True)
class PrivateKey:
    """Private key with curve parameters."""

    curve: EllipticCurve
    d: int

@dataclass(frozen=True)
class PublicKey:
    """Public key with curve parameters."""

    curve: EllipticCurve
    q_point: ECPoint

def _parse_point(raw: Any, field_name: str) -> ECPoint:
    """Parse point from JSON as {'x': int, 'y': int} or [x, y]."""
    if isinstance(raw, dict):
        return ECPoint(int(raw["x"]), int(raw["y"]))
    if isinstance(raw, list) and len(raw) == 2:
        return ECPoint(int(raw[0]), int(raw[1]))
    raise ValueError(f"Field '{field_name}' must be point object
or [x, y]")
```

```

def _curve_from_key_json(key_data: dict[str, Any]) ->
EllipticCurve:
    """Build curve object from key JSON fields."""
    base_point = _parse_point(key_data["P"], "P")
    curve_params = CurveParams(
        curve_id=str(key_data.get("curve_id",
APP_CONFIG.default_curve_id)),
        p=int(key_data["p"]),
        a=int(key_data["a"]),
        b=int(key_data["b"]),
        q=int(key_data["q"]),
        px=base_point.x,
        py=base_point.y,
    )
    curve = EllipticCurve(
        p=curve_params.p,
        a=curve_params.a,
        b=curve_params.b,
        q=curve_params.q,
        base_point=base_point,
        curve_id=curve_params.curve_id,
    )
    if not curve.is_on_curve(curve.base_point):
        raise ValueError("Base point P is not on the curve")
    if curve.base_point.is_infinity:
        raise ValueError("Base point P must not be infinity")
    if not curve.scalar_mul(curve.q,
curve.base_point).is_infinity:
        raise ValueError("Base point P must have order q")
    return curve

def load_private_key(path: Path) -> PrivateKey:
    """Load private key from JSON file."""
    data = json.loads(path.read_text(encoding="utf-8"))
    curve = _curve_from_key_json(data)
    d = int(data["d"])
    if not (0 < d < curve.q):
        raise ValueError("Private key d must satisfy 0 < d < q")
    return PrivateKey(curve=curve, d=d)

def load_public_key(path: Path) -> PublicKey:
    """Load public key from JSON file."""
    data = json.loads(path.read_text(encoding="utf-8"))
    curve = _curve_from_key_json(data)

```

```

q_point = _parse_point(data["Q"], "Q")
if not curve.is_on_curve(q_point):
    raise ValueError("Public key point Q is not on the curve")
if q_point.is_infinity:
    raise ValueError("Public key point Q must not be infinity")
if not curve.scalar_mul(curve.q, q_point).is_infinity:
    raise ValueError(
        "Public key point Q must belong to subgroup of order
q")
return PublicKey(curve=curve, q_point=q_point)

def _compute_e(file_path: Path, hash_alg: HashAlg, q: int) -> int:
    """Compute e = alpha mod q, where alpha = h(M)."""
    alpha = hash_to_alpha(file_path, hash_alg)
    e = alpha % q
    return 1 if e == 0 else e

def sign_file(
    file_path: Path,
    private_key_path: Path,
    signature_path: Path,
    hash_alg: HashAlg = cast(HashAlg,
APP_CONFIG.default_hash_alg),
) -> SignaturePayload:
    """Create signature and save it in temporary binary
container."""
    private_key = load_private_key(private_key_path)
    curve = private_key.curve
    e = _compute_e(file_path=file_path, hash_alg=hash_alg,
q=curve.q)

    # Signature generation repeats until both r and s are non-
zero.
    while True:
        k = secrets.randbelow(curve.q - 1) + 1 # random k in (0,
q)

        c_point = curve.scalar_mul(k, curve.base_point)
        if c_point.is_infinity:
            continue

        r = c_point.x % curve.q
        if r == 0:
            continue

```

```

        s = (r * private_key.d + k * e) % curve.q
        if s == 0:
            continue
        break
    q_point = curve.scalar_mul(private_key.d, curve.base_point)
    if q_point.is_infinity:
        raise ValueError("Derived public key Q is infinity")
    payload = SignaturePayload(
        hash_alg=hash_alg,
        curve_id=curve.curve_id,
        qx=q_point.x,
        qy=q_point.y,
        p=curve.p,
        a=curve.a,
        b=curve.b,
        px=curve.base_point.x,
        py=curve.base_point.y,
        q=curve.q,
        r=r,
        s=s,
    )
    save_signature(signature_path, payload)
    return payload

def verify_file(
    file_path: Path,
    signature_path: Path,
    public_key_path: Path,
) -> bool:
    """Verify signature according to educational GOST R 34.10-2018
    steps."""
    try:
        public_key = load_public_key(public_key_path)
        payload = load_signature(signature_path)
    except (ValueError, json.JSONDecodeError, OSError):
        return False

    curve = public_key.curve

    if payload.qx != public_key.q_point.x or payload.qy !=
public_key.q_point.y:
        return False

    if (
        payload.p != curve.p

```

```

        or payload.a != curve.a
        or payload.b != curve.b
        or payload.px != curve.base_point.x
        or payload.py != curve.base_point.y
        or payload.q != curve.q
    ):
        return False

    if payload.hash_alg not in ("streebog256", "streebog512"):
        return False
    hash_alg = cast(HashAlg, payload.hash_alg)

    r = payload.r
    s = payload.s
    if not (0 < r < curve.q and 0 < s < curve.q):
        return False

    try:
        e = _compute_e(file_path=file_path, hash_alg=hash_alg,
q=curve.q)
        v = mod_inv(e, curve.q)
    except (ValueError, OSError):
        return False

    z1 = (s * v) % curve.q
    z2 = (-r * v) % curve.q

    c_point = curve.add(
        curve.scalar_mul(z1, curve.base_point),
        curve.scalar_mul(z2, public_key.q_point),
    )
    if c_point.is_infinity:
        return False

    r_check = c_point.x % curve.q
    return r_check == r

```

ПРИЛОЖЕНИЕ 5. ЛИСТИНГ ФАЙЛА SRC/MAIN.PY

```
"""CLI entry point for file signing and signature verification."""

from __future__ import annotations

import argparse
import sys
from pathlib import Path

from src.config import APP_CONFIG
from src.sign import sign_file, verify_file

def build_parser() -> argparse.ArgumentParser:
    """Create CLI parser with sign and verify commands."""
    parser = argparse.ArgumentParser(
        prog="gost-sign-cli",
        description="Educational signer/verifier for GOST R 34.10-2018",
    )
    subparsers = parser.add_subparsers(dest="command", required=True)

    sign_parser = subparsers.add_parser("sign", help="Sign file")
    sign_parser.add_argument("--file", required=True, type=Path, help="Input file path")
    sign_parser.add_argument(
        "--private-key", required=True, type=Path, help="Path to private key JSON"
    )
    sign_parser.add_argument(
        "--signature", required=True, type=Path, help="Output signature file path"
    )
    sign_parser.add_argument(
        "--hash-alg",
        default=APP_CONFIG.default_hash_alg,
        choices=["streebog256", "streebog512"],
        help="Hash algorithm",
    )

    verify_parser = subparsers.add_parser("verify", help="Verify file signature")
    verify_parser.add_argument("--file", required=True, type=Path, help="Input file path")
```

```

        verify_parser.add_argument(
            "--signature", required=True, type=Path, help="Path to
signature file"
        )
        verify_parser.add_argument(
            "--public-key", required=True, type=Path, help="Path to
public key JSON"
        )

    return parser

def main(argv: list[str] | None = None) -> int:
    """Run CLI command and return process exit code."""
    parser = build_parser()
    args = parser.parse_args(argv)

    if args.command == "sign":
        try:
            sign_file(
                file_path=args.file,
                private_key_path=args.private_key,
                signature_path=args.signature,
                hash_alg=args.hash_alg,
            )
        except (ValueError, OSError) as exc:
            print(f"SIGN ERROR: {exc}")
            return 1
        print(f"Signature saved to: {args.signature}")
        return 0

    if args.command == "verify":
        is_valid = verify_file(
            file_path=args.file,
            signature_path=args.signature,
            public_key_path=args.public_key,
        )
        print("VALID" if is_valid else "INVALID")
        return 0 if is_valid else 1

    parser.error("Unknown command")
    return 2

if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))

```

ПРИЛОЖЕНИЕ 6. ЛИСТИНГ ФАЙЛА TESTS/TEST_SMOKE.PY

```
"""Smoke tests for sign/verify flow."""

from __future__ import annotations

import json
from pathlib import Path
from src.sign import sign_file, verify_file
from src.ecc import ECPoint, EllipticCurve
from src.sigfile import load_signature, save_signature,
SignaturePayload

def _write_test_keys(tmp_path: Path) -> tuple[Path, Path]:
    # Маленькая учебная кривая для smoke-тестов.
    # Оставляем её отдельной от реального варианта 8, чтобы тесты
    # были быстрыми.
    p = 17
    a = 2
    b = 2
    q = 19
    px = 5
    py = 1
    d = 7

    curve = EllipticCurve(
        p=p,
        a=a,
        b=b,
        q=q,
        base_point=ECPoint(px, py),
        curve_id="edu-tiny-v1",
    )

    q_point = curve.scalar_mul(d, curve.base_point)

    private_key = {
        "curve_id": "edu-tiny-v1",
        "p": p,
        "a": a,
        "b": b,
        "q": q,
        "P": {"x": px, "y": py},
        "d": d,
    }
```



```

public_key = {
    "curve_id": "edu-tiny-v1",
    "p": p,
    "a": a,
    "b": b,
    "q": q,
    "P": {"x": px, "y": py},
    "Q": {"x": q_point.x, "y": q_point.y},
}

private_key_path = tmp_path / "private.json"
public_key_path = tmp_path / "public.json"

private_key_path.write_text(json.dumps(
    private_key, indent=2), encoding="utf-8")
public_key_path.write_text(json.dumps(
    public_key, indent=2), encoding="utf-8")

return private_key_path, public_key_path

def test_sign_and_verify_success(tmp_path: Path) -> None:
    """Smoke: generated signature must verify for original file."""
    private_key_path, public_key_path = _write_test_keys(tmp_path)
    message_path = tmp_path / "message.txt"
    signature_path = tmp_path / "message.sig"
    message_path.write_text("KMZI    Lab    6    test    message",
encoding="utf-8")

    sign_file(message_path, private_key_path, signature_path)
    assert verify_file(message_path, signature_path,
public_key_path) is True

def test_verify_fails_for_modified_message(tmp_path: Path) ->
None:
    """Smoke: verification must fail after message
modification."""
    private_key_path, public_key_path = _write_test_keys(tmp_path)
    message_path = tmp_path / "message.txt"
    signature_path = tmp_path / "message.sig"
    message_path.write_text("original", encoding="utf-8")

    sign_file(message_path, private_key_path, signature_path)
    message_path.write_text("tampered", encoding="utf-8")

```

```

        assert          verify_file(message_path,          signature_path,
public_key_path) is False

def test_verify_fails_for_corrupted_signature(tmp_path: Path) ->
None:
    """Smoke: verification must fail for corrupted signature
values."""
    private_key_path, public_key_path = _write_test_keys(tmp_path)
    message_path = tmp_path / "message.txt"
    signature_path = tmp_path / "message.sig"
    message_path.write_text("message", encoding="utf-8")

    sign_file(message_path, private_key_path, signature_path)

    payload = load_signature(signature_path)

    # Портим именно значение подписи, а не случайный байт ASN.1-
контейнера.
    bad_s = payload.s + 1
    if bad_s == payload.q:
        bad_s = 1

    corrupted_payload = SignaturePayload(
        hash_alg=payload.hash_alg,
        curve_id=payload.curve_id,
        qx=payload.qx,
        qy=payload.qy,
        p=payload.p,
        a=payload.a,
        b=payload.b,
        px=payload.px,
        py=payload.py,
        q=payload.q,
        r=payload.r,
        s=bad_s,
    )
    save_signature(signature_path, corrupted_payload)

    assert          verify_file(message_path,          signature_path,
public_key_path) is False

```

ПРИЛОЖЕНИЕ 7. ЛИСТИНГ ФАЙЛА SRC/CONFIG.PY

```
from dataclasses import dataclass

@dataclass(frozen=True)
class CurveParams:
    """Elliptic curve parameters in short Weierstrass form."""

    curve_id: str
    p: int
    a: int
    b: int
    q: int
    px: int
    py: int

@dataclass(frozen=True)
class AppConfig:
    """Global configuration for signing and verification."""

    default_curve_id: str
    default_hash_alg: str
    gost_algorithm_id: bytes
    signature_key_label: str

# Вариант 8 из Приложения Д.
# В методичке порядок группы обозначен как r, а в коде мы используем
поле q.
# Это один и тот же параметр порядка подгруппы.
VARIANT_8 = CurveParams(
    curve_id="gost3410-2018-var8",

    p=578960446289587186312130282755184113284761495997897707387572188
    40632915517411,
    a=-1,

    b=445164239480196618254208139279655923416758392708498604418610206
    78502941837466,

    q=289480223144793593156065141377592056642368320232316280358711934
    93020981068937,
```

```
px=23235760586019566647209667080457263089166278247417077298367088
87517232685058,

py=20772302011053991390127435262297715010367018383131467831609444
907978987653753,
)

CURVE_VARIANTS: dict[str, CurveParams] = {
    VARIANT_8.curve_id: VARIANT_8,
}

APP_CONFIG = AppConfig(
    default_curve_id=VARIANT_8.curve_id,
    default_hash_alg="streebog256",
    gost_algorithm_id=bytes.fromhex("80060700"),
    signature_key_label="gostSignKey",
)
```